

Semantic Monitoring for Adaptive Compute Allocation in Edge Robotic Vision-Language Systems

*A thesis submitted in partial fulfillment of the requirements
for the award of the degree of*

MASTER OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

with specialization in

HUMAN COMPUTER INTERACTION



Submitted by

DEVARAKONDA SHANMUKHA SAI

(MHC2024013)

Under the Supervision of

PROF. G. C. NANDI

Submitted to

DEPARTMENT OF INFORMATION TECHNOLOGY

भारतीय सूचना प्रौद्योगिकी संस्थान, इलाहाबाद

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, ALLAHABAD

June 2026



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

SUPERVISOR CERTIFICATE

It is certified that the work contained in the thesis titled “**Semantic Monitoring for Adaptive Compute Allocation in Edge Robotic Vision-Language Systems**”, submitted by **Mr. Devarakonda Shanmukha Sai**, Roll No. **MHC2024013**, has been carried out under my/our supervision.

The work embodied in this thesis has not been submitted elsewhere for the award of any degree.

Prof. G. C. Nandi

Department of Information Technology

Indian Institute of Information Technology, Allahabad



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CANDIDATE DECLARATION

I, **Devarakonda Shanmukha Sai**, Roll No. **MHC2024013**, certify that this thesis titled “**Semantic Monitoring for Adaptive Compute Allocation in Edge Robotic Vision-Language Systems**” is submitted by me towards partial fulfillment of the requirements for the award of the degree of **Master of Technology in Information Technology** with specialization in **Human Computer Interaction, Department of Information Technology, Indian Institute of Information Technology, Allahabad.**

I understand that plagiarism includes:

1. Reproducing someone else's work (fully or partially) or ideas and claiming it as one's own.
2. Reproducing someone else's work (verbatim copying or paraphrasing) without crediting the original source.
3. Committing literary theft (copying some unique literary construct).

I have given due credit to the original authors and sources through proper citation for all the words, ideas, diagrams, graphics, computer programs, experiments, results, and websites that are not my original contributions. I have used quotation marks to identify verbatim sentences and given due credit to the original authors and sources.

I affirm that no portion of my work is plagiarized. In the event of a complaint of plagiarism, I shall be fully responsible. I understand that my supervisor may not be in a position to verify that this work is not plagiarized.

Devarakonda Shanmukha Sai
MHC2024013
Department of Information Technology
Indian Institute of Information Technology,
Allahabad
Prayagraj – 211015, Uttar Pradesh, India

Date: _____



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament
Deoghat Jhalwa, Prayagraj 211015 (U.P.) India
Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CERTIFICATE OF APPROVAL

This is to certify that the thesis entitled “**Semantic Monitoring for Adaptive Compute Allocation in Edge Robotic Vision-Language Systems**”, submitted by **Mr. Devarakonda Shanmukha Sai**, Roll No. **MHC2024013**, in partial fulfillment of the requirements for the award of the degree of **Master of Technology in Information Technology** with specialization in **Human Computer Interaction** at **Indian Institute of Information Technology, Allahabad**, has been examined and approved by the Thesis Evaluation Board.

The work embodied in this thesis is found to be satisfactory and is approved for the award of the degree of **Master of Technology**. It is understood that by this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approve the thesis only for the purpose for which it is submitted.

Date: _____

Place: _____

Prof. G. C. Nandi

Chairperson, Thesis Evaluation Board

Signature: _____

Co-Supervisor (if any)

Signature: _____

Internal Examiner Name

Internal Examiner

Signature: _____

External Examiner Name, Affiliation

External Examiner

Signature: _____

Prof. Manish Kumar

Head of Department / Program Coordinator

Signature: _____

Prof. Pavan Chakraborty

Dean Academic Affairs

Signature: _____

Department of Information Technology
Indian Institute of Information Technology, Allahabad
Prayagraj – 211015, Uttar Pradesh, India

Semantic Monitoring for Adaptive Compute Allocation in Edge Robotic Vision-Language Systems

ABSTRACT

Master of Technology

Department of Information Technology

by Devarakonda Shanmukha Sai

Vision-language models (VLMs) endow edge robots with open-vocabulary scene understanding, but their per-inference cost in compute, latency, and energy is prohibitive on resource-constrained accelerators. Mainstream robotic pipelines invoke the VLM on every frame, or on a fixed sampling schedule, regardless of whether the scene has meaningfully changed. Because consecutive frames in a continuous video stream are strongly correlated, this discipline repeatedly re-derives the same understanding of an unchanging scene — an inefficiency this thesis names *semantic reasoning redundancy*, distinct from the pixel-, frame-, and parameter-level redundancies targeted by prior efficiency research. This thesis proposes a *monitor-then-allocate* architecture in which an always-on, lightweight monitor maintains a composite semantic state from cheap object-detection, multi-object-tracking, and image-embedding signals, and estimates a bounded scalar drift signal $D_t = \alpha D_{\text{embed}} + \beta D_{\text{objects}} + \gamma D_{\text{track}}$ that fuses embedding distance, object-set novelty, and tracking-identity churn against a temporally decayed memory. A three-tier policy maps this signal through two interpretable thresholds to cache reuse (no VLM), lightweight reasoning, or full reasoning, so that expensive multimodal compute is spent only when the scene genuinely changes. The reasoning model is treated as a replaceable black box and the policy is training-free. A modular, edge-oriented reference implementation integrating YOLOv8-n, ByteTrack, CLIP ViT-B/32, and Moondream2 is described, and the trade-off between compute saved and semantic fidelity retained is quantified with a unified *Semantic Compute Efficiency* (SCE) metric against every-frame, uniform-sampling, motion-gating, and embedding-only baselines. In a preliminary evaluation on a continuous stream of 56,314 frames, the proposed three-tier scheduler resolved over 90% of frames from the semantic cache, reducing vision-language model invocations to 9.8% of frames at high semantic retention.

Keywords: Vision-language models · Edge robotics · Adaptive compute allocation · Semantic drift · Temporal semantic memory · Selective inference · Object tracking · CLIP embeddings

Acknowledgements

I express my sincere gratitude to my thesis supervisor, **Prof. G. C. Nandi**, for their invaluable guidance, constructive feedback, and continuous support throughout the course of this research. Their deep expertise in machine learning and computer vision shaped the technical direction of this work, and their encouragement during challenging phases of experimentation was instrumental in its completion.

I am grateful to the faculty and staff of the **Department of Information Technology, Indian Institute of Information Technology, Allahabad**, for providing an excellent research environment and computational resources that made this work possible.

I thank the open-source communities behind PyTorch, Ultralytics YOLO, ByteTrack, CLIP, and Moondream, whose tools and reference implementations enabled the system and the fair, reproducible comparisons reported in this thesis. The broader research community whose foundational work on vision-language modelling, efficient inference, and embodied perception directly enabled the contributions of this thesis deserves acknowledgement.

Finally, I am deeply grateful to my family and friends for their unwavering support, patience, and encouragement throughout the M.Tech programme.

Contents

Abstract	iv
Acknowledgements	vi
Contents	vii
List of Figures	x
List of Tables	xi
List of Abbreviations	xii
List of Symbols	xiii
1 Introduction	1
1.1 Background and Contemporary Context	1
1.2 The Core Problem: Semantic Reasoning Redundancy	2
1.3 Research Motivation and the Need for Semantic-Aware Scheduling	3
1.4 Statement of Core Research Objectives	4
1.5 Summary of Scientific Contributions	5
1.6 Thesis Document Organization	6
2 Literature Review	7
2.1 Vision-Language Models	7
2.2 Vision-Language Models in Robotics	8
2.3 Efficient and Edge Inference	8
2.4 Temporal Redundancy Reduction in Video	9
2.5 Object Detection and Multi-Object Tracking	9
2.6 Semantic Representations and Concept Drift	10
2.7 Adaptive and Conditional Computation	10
2.8 Efficient Video Analytics Systems	10
2.9 Streaming and Adaptive Multimodal Inference	11
2.10 Comparative Positioning	11

3	Proposed Methodology	13
3.1	Overview and Design Principles	13
3.2	Semantic State Representation	15
3.3	Temporal Semantic Memory	15
3.4	Semantic Drift Estimation	16
3.5	Egocentric Operation and Camera-Motion Compensation	17
3.6	Adaptive Compute-Allocation Policy	18
3.7	Selective Invocation and Semantic Caching	19
3.8	Semantic Compute Efficiency	19
3.9	The Semantic Observatory and Policy-Discovery Methodology	20
4	Software Architecture and Engineering Design	22
4.1	System Architecture Overview	22
4.2	Component-Level Implementation Breakdown	22
4.2.1	Frame Filtering and Detection	22
4.2.2	Tracking and Embedding	23
4.2.3	Temporal Memory and Drift Estimation	23
4.2.4	Scheduler, Cache, and VLM Wrapper	24
4.2.5	Vision-Language Model Backends	24
4.3	Control Flow and the Policy Abstraction	25
4.4	Hyperparameter Configuration	26
4.5	The Semantic Observatory Subsystem	26
4.6	Hardware Optimization and Execution Strategy	27
5	Experimental Setup and Empirical Evaluation	28
5.1	Video Environment and Parameters	28
5.2	Formal Evaluation Metrics	29
5.3	Baseline Policies	29
5.4	Quantitative Performance Analysis	30
5.4.1	Comparison Against Baselines	31
5.5	Cross-Domain Applicability and Invocation Patterns	32
5.6	Planned Ablation Studies	34
5.7	Discussion	35
6	Conclusion and Future Scope	36
6.1	Thesis Synthesis and Final Review	36
6.2	Architectural Trade-offs and Computational Costs	37

6.3	Directions for Future Research	38
	References	40
A	Operational Guide and Reproducibility Manual	44
A.1	Hardware and System Prerequisites	44
A.2	Environment Assembly and Dependency Registry	44
A.3	Command-Line Execution	45
A.3.1	Running all baselines and the proposed scheduler	45
A.3.2	Individual runs and the threshold sweep	45
A.4	Output Structure	46

List of Figures

3.1	System architecture	14
5.1	Tier distribution of the proposed run	31
5.2	Drift signal and thresholds	32
5.3	Expected threshold sweep	35

List of Tables

2.1	Comparison of efficiency approaches	12
3.1	Semantic Observatory signals	20
4.1	Source module layout	23
4.2	Supported VLM backends	25
4.3	Default hyperparameters	26
5.1	Measured proposed-scheduler performance	31
5.2	Main quantitative comparison	33
5.3	Cross-domain invocation patterns	33
A.1	Dependency registry	45

List of Abbreviations

VLM	Vision-Language Model
VQA	Visual Question Answering
CLIP	Contrastive Language-Image Pre-training
YOLO	You Only Look Once (object detector)
ViT	Vision Transformer
MOT	Multi-Object Tracking
SCE	Semantic Compute Efficiency
FPS	Frames Per Second
GPU	Graphics Processing Unit
CPU	Central Processing Unit
CNN	Convolutional Neural Network
FLOPs	Floating-Point Operations
IoU	Intersection over Union
COCO	Common Objects in Context (dataset)
CUDA	Compute Unified Device Architecture

List of Symbols

Z_t	Composite Semantic State at Frame t	Fused Descriptor
E_t	CLIP Image Embedding (unit-norm)	$\mathbb{R}^d$
O_t	Object Class–Count Multiset	Symbolic Set
$\mathcal{T}_t$	Active Track Identity Set	Identity Set
M_t	Temporal Semantic Memory	$\mathbb{R}^d$
D_t	Composite Semantic Drift	$[0, 1]$
D_{embed}	Embedding (cosine) Drift	$[0, 1]$
D_{objects}	Object-Set Novelty	$[0, 1]$
D_{track}	Tracking-Identity Churn	$[0, 1]$
α, β, γ	Drift Component Weights ($\sum = 1$)	Convex Weights
$\tau_{\text{low}}, \tau_{\text{high}}$	Tier Decision Thresholds	$[0, 1]$
λ	Memory Decay Factor	$(0, 1)$
c_t	Camera-Motion (Egomotion) Fraction	$[0, 1]$
SCE	Semantic Compute Efficiency	Real Scalar

Chapter 1

Introduction

This chapter situates the thesis within the rapidly evolving landscape of embodied artificial intelligence, where vision-language models are increasingly deployed on resource-constrained robotic platforms. It identifies the central inefficiency that motivates this work — the repeated invocation of expensive multimodal reasoning even when the semantic content of a scene is essentially unchanged — and frames the research question, objectives, and contributions around the reduction of this semantic reasoning redundancy.

1.1 Background and Contemporary Context

The convergence of large-scale vision-language models (VLMs) with mobile robotic platforms has opened a new frontier in embodied perception. Models such as CLIP [1], BLIP-2 [2], and LLaVA [3] couple visual encoders with language reasoning to produce open-vocabulary scene descriptions, answer questions about an environment, and ground natural-language instructions in physical affordances. When integrated into robotic control stacks — as in RT-2 [4], SayCan [5], and OpenVLA [6] — these models allow a robot to interpret its surroundings with a flexibility that hand-engineered perception pipelines cannot match.

This expressive power, however, carries a substantial computational price. A single forward pass through a modern VLM may involve billions of floating-point operations, hundreds of milliseconds of latency, and several gigabytes of accelerator memory. On the consumer-grade and embedded accelerators typical of edge robots — as opposed to the data-centre GPUs on which these models are trained — such costs translate directly into

reduced battery life, thermal throttling, and an inability to maintain real-time perception rates. The tension between the semantic richness of VLMs and the strict energy, latency, and memory envelopes of edge robotics is the defining systems challenge addressed by this thesis.

Crucially, a robot perceiving a continuous video stream does not encounter a sequence of independent, equally informative images. Consecutive frames are strongly correlated: a robot pausing in a corridor, waiting for a door to open, or observing a static workspace produces long runs of frames whose *semantic* content barely changes even as raw pixels fluctuate with sensor noise, illumination, and minor camera motion. The opportunity that this thesis exploits is the gap between *pixel-level* change and *semantic-level* change: the former is frequent and cheap to measure, whereas the latter is rare and is what actually warrants fresh multimodal reasoning.

1.2 The Core Problem: Semantic Reasoning Redundancy

Contemporary robotic VLM systems overwhelmingly adopt a *stateless, per-frame* invocation discipline: each incoming frame, or every K -th frame under a fixed sampling schedule, is passed to the VLM regardless of whether the scene has meaningfully evolved since the last invocation. This design is simple and reproducible, but it is profoundly wasteful. When a robot stares at an unchanging scene for several seconds, a per-frame policy re-derives the same caption or answer dozens of times, consuming accelerator cycles and energy to produce information the system already possesses.

We term this inefficiency *semantic reasoning redundancy*: the repeated expenditure of heavyweight multimodal compute on inputs whose semantic state is, for all practical purposes, identical to a previously reasoned-over state. It is conceptually distinct from the redundancies that prior efficiency research targets. Pixel- and frame-level redundancy — addressed by adaptive frame sampling [7], salient-clip selection [8], and skip convolutions [9] — concerns whether the raw signal has changed. Parameter redundancy —

addressed by quantization [10], pruning and compression [11], and distillation [12] — concerns whether a model is larger than it needs to be. Neither asks the question at the heart of this work: *given what the system already understands about the scene, does the next frame justify the cost of multimodal reasoning at all?*

Answering this question requires the system to maintain an explicit, continuously updated notion of *semantic state*, to quantify how far the current observation has drifted from that state, and to allocate compute in proportion to that drift. A scene that is semantically stable should be served almost for free from memory; a scene undergoing a genuine semantic transition — a new object entering view, an agent leaving the frame, a task-relevant change in configuration — should trigger full reasoning. The absence of such a mechanism in mainstream pipelines is the gap this thesis fills.

1.3 Research Motivation and the Need for Semantic-Aware Scheduling

Three observations motivate a semantic-aware scheduling approach.

Observation 1: Semantic change is bursty, not uniform. In realistic robotic deployments, semantically significant events are sparse and clustered in time, separated by long quiescent intervals. A compute policy that spends a fixed budget per frame is therefore mismatched to the underlying signal: it over-provisions during quiescence and may under-provision during bursts. A policy that tracks semantic drift can shift budget from the former to the latter.

Observation 2: Cheap signals can predict the value of expensive reasoning. Lightweight perception primitives that already run on edge robots — a nano-scale object detector [13], a multi-object tracker [14], and a compact image embedding [1] — jointly summarise the scene at a small fraction of a VLM’s cost. Changes in these cheap signals are strong proxies for the kind of semantic change that warrants invoking the VLM. The

cost of *deciding* whether to reason can thus be made far smaller than the cost of reasoning itself.

Observation 3: Most useful outputs are reusable. Because the semantic state changes slowly, the textual output of a VLM remains valid for the duration of a stable interval. Caching the most recent output and reusing it while drift remains below a threshold preserves the information delivered to downstream robotic tasks while eliminating redundant computation.

Taken together, these observations argue for a *monitor-then-allocate* architecture: a lightweight semantic monitor runs every frame, estimates a scalar drift signal, and a scheduling policy maps that signal to a discrete compute tier ranging from cache reuse (no VLM) to full multimodal reasoning. This thesis designs, implements, and evaluates such an architecture.

1.4 Statement of Core Research Objectives

The work is organised around the following research question:

How can semantic state transitions be used to dynamically control multimodal compute allocation in edge robotic vision systems, while preserving the semantic fidelity of the perception delivered to downstream tasks?

This question is decomposed into four concrete objectives:

1. **Formalise semantic state and semantic drift.** Define a composite semantic-state representation from cheap per-frame signals (embedding, object metadata, tracking identities) and a corresponding scalar drift measure that is bounded, interpretable, and sensitive to task-relevant change.
2. **Design an adaptive compute-allocation policy.** Map the drift signal onto a small set of compute tiers via interpretable thresholds, such that compute is spent in proportion to semantic novelty.

3. **Implement an edge-deployable pipeline.** Engineer a modular system that integrates frame filtering, detection, tracking, embedding, temporal memory, drift estimation, scheduling, caching, and selective VLM invocation, suitable for execution on commodity accelerators.
4. **Evaluate against principled baselines.** Quantify the trade-off between compute saved and semantic fidelity retained, comparing the proposed scheduler against every-frame, uniform-sampling, motion-gating, and embedding-only policies using a unified metric of *semantic compute efficiency*.

1.5 Summary of Scientific Contributions

The principal contributions of this thesis are:

- **A formulation of semantic reasoning redundancy** as a first-class efficiency target for edge VLM systems, distinct from the pixel-, frame-, and parameter-level redundancies addressed by prior work (Chapter 2).
- **A composite semantic-drift model** $D_t = \alpha D_{\text{embed}} + \beta D_{\text{objects}} + \gamma D_{\text{track}}$ that fuses embedding distance, object-set novelty, and tracking-identity change into a single bounded signal, together with a temporal semantic memory that suppresses spurious drift from sensor noise (Chapter 3).
- **A three-tier adaptive compute-allocation policy** that maps drift to cache reuse, lightweight reasoning, or full reasoning through interpretable thresholds τ_{low} and τ_{high} (Chapter 3).
- **A modular, edge-oriented reference implementation** and the *Semantic Compute Efficiency* (SCE) metric that normalises retained semantic fidelity by compute cost, enabling fair comparison across policies (Chapters 4 and 5).
- **A Semantic Observatory and a methodology for discovering scheduling policies.** Rather than positing a single hand-tuned heuristic, this thesis contributes an instrument — the Semantic Observatory — that derives a suite of semantic-evolution

signals (drift, velocity, acceleration, volatility, scene entropy, object persistence, stability and transition regions) from a pipeline run, and a disciplined observe–hypothesise–modify-one-thing–evaluate protocol that uses these signals to *discover and validate* scheduling policies from evidence (Sections 3.9 and 4.5). The scientific contribution is thus a methodology for studying semantic evolution and deriving policies, not merely one scheduler.

1.6 Thesis Document Organization

The remainder of this thesis is organised as follows. Chapter 2 reviews vision-language modelling, efficient and edge inference, temporal redundancy reduction, object detection and tracking, and adaptive computation, positioning the present work against each. Chapter 3 develops the proposed methodology: the semantic-state representation, temporal memory, drift estimator, tiered scheduling policy, and the SCE metric. Chapter 4 describes the software architecture and engineering design, detailing each component and the per-frame control flow. Chapter 5 presents the experimental setup, baselines, evaluation metrics, quantitative results, and an ablation study. Chapter 6 synthesises the findings, discusses trade-offs and limitations, and outlines directions for future research. Appendix A provides a reproducibility manual.

Chapter 2

Literature Review

This chapter surveys the research streams on which the thesis builds: vision-language modelling, efficient and edge inference, temporal redundancy reduction in video, real-time object detection and multi-object tracking, semantic representations and concept drift, and adaptive conditional computation. Each section closes by identifying the gap that motivates a semantic-aware scheduling layer, and the chapter ends with a comparative positioning of the proposed approach.

2.1 Vision-Language Models

Vision-language models learn a joint representation of images and text that supports open-vocabulary recognition and reasoning. CLIP [1] established the contrastive pre-training paradigm, aligning an image encoder and a text encoder in a shared embedding space using large-scale web data; its normalised embeddings have since become a de-facto semantic descriptor for images. Generative VLMs extend this paradigm to free-form language output: BLIP-2 [2] bridges a frozen vision encoder to a frozen large language model through a lightweight querying transformer, while LLaVA [3] fine-tunes a projection between visual features and an instruction-tuned language model. These architectures inherit the transformer backbone [15, 16] and, with it, a per-inference cost that scales with input resolution and sequence length.

For edge deployment, compact VLMs are essential. Moondream [17] is a representative tiny VLM (on the order of a few billion parameters) designed for captioning and visual question answering on modest hardware. Even so, a single Moondream invocation

remains orders of magnitude more expensive than the lightweight perception primitives discussed below, which is precisely why *when* to invoke it matters.

2.2 Vision-Language Models in Robotics

A growing body of work embeds VLMs into robotic control. SayCan [5] grounds language-model plans in learned robotic affordances; RT-2 [4] and OpenVLA [6] train vision-language-action models that map observations and instructions directly to control. Closer to the temporal setting of this thesis, agentic systems increasingly maintain a memory of the scene rather than reasoning from each observation in isolation: VideoAgent [18] couples a multimodal agent with an explicit memory to answer queries over long videos, and VideoARM [19] performs agentic reasoning over a hierarchical memory for long-form video understanding. These systems demonstrate the value of multimodal reasoning and persistent semantic memory for understanding continuous video, but they typically assume access to substantial compute and treat perception as a per-step query. None incorporates an explicit notion of *semantic stability* that would let the agent skip reasoning when its understanding of the scene is unchanged — the omission this thesis targets.

2.3 Efficient and Edge Inference

Reducing the cost of neural inference is a mature field. Architectural efficiency is pursued through compact backbones such as MobileNets [20]. Post-training compression includes pruning and quantization [11] and low-precision matrix multiplication tailored to transformers [10]. Knowledge distillation [12] transfers the behaviour of a large model into a smaller one; TinyCLIP [21] applies this idea to the embedding model itself, distilling a compact CLIP suitable for edge use — directly relevant to the lightweight embedding the monitor relies on. The broader case for moving inference to the network edge, and the energy, latency, and bandwidth constraints that motivate it, is set out in surveys of edge intelligence [22]. These techniques reduce the cost of an *individual* forward pass, and are complementary to this thesis: they make each VLM call cheaper, whereas the proposed

scheduler reduces the *number* of calls. The two axes multiply, and a complete edge system would exploit both.

2.4 Temporal Redundancy Reduction in Video

Because video is temporally redundant, classical recognition pipelines first established that sparse temporal sampling suffices for many tasks: temporal segment networks [23] recognise actions from a handful of sparsely sampled snippets, and attention-based video transformers [24] model temporal structure across frames. Building on this, several methods learn *which* frames deserve expensive processing: AdaFrame [7] trains a policy to select informative frames for action recognition; SCSampler [8] samples salient clips using a cheap proxy; skip-convolutions [9] avoid recomputing convolutional features in regions that do not change between frames. These approaches operate at the level of raw appearance or low-level features and are usually trained end-to-end for a fixed downstream task. The present work differs in two respects: its trigger is a *semantic* drift signal rather than appearance change, and it is *training-free* at the policy level, relying on interpretable thresholds rather than a learned sampler, which suits the heterogeneous, label-scarce conditions of robotic deployment.

2.5 Object Detection and Multi-Object Tracking

Real-time object detection has been driven by single-stage detectors since YOLO [25]; the YOLOv8 family [13] offers nano-scale variants that run comfortably on edge accelerators while reporting object classes and bounding boxes trained on COCO [26]. Multi-object tracking associates detections across frames to maintain stable identities: SORT [27] couples a Kalman filter with Hungarian assignment, and ByteTrack [14] improves robustness by associating low-confidence boxes as well. In this thesis these components are not contributions in themselves; rather, their outputs — object classes, counts, and identity continuity — are repurposed as cheap, informative features for estimating semantic drift.

2.6 Semantic Representations and Concept Drift

Treating the evolving content of a video stream as a non-stationary signal connects this work to the literature on *concept drift*, surveyed comprehensively by Gama et al. [28]. There, drift detection governs when a learned model should be updated; here, an analogous drift signal governs when expensive reasoning should be re-invoked. The semantic descriptor we monitor is the CLIP embedding [1], whose cosine geometry provides a natural, bounded distance between scene states. The novelty of the present formulation is to *fuse* embedding drift with symbolic object-set and tracking-identity drift into one decision signal, rather than relying on a single modality.

2.7 Adaptive and Conditional Computation

Conditional computation allocates network capacity per input. Early-exit networks such as BranchyNet [29] terminate inference once an intermediate classifier is confident; adaptive computation time [30] learns how many steps to spend per input; and dynamic routing methods such as SkipNet [31] skip layers conditionally. The survey of Han et al. [32] organises these into sample-wise, spatial-wise, and temporal-wise dynamics. The proposed scheduler is a temporal-wise dynamic mechanism operating at the granularity of *whole-model invocation across frames*, controlled by an external semantic monitor rather than by internal model confidence — a design that keeps the expensive VLM a black box and is therefore agnostic to the specific model used.

2.8 Efficient Video Analytics Systems

A systems-oriented line of research reduces the cost of querying neural networks over video streams. NoScope [33] accelerates queries by learning cheap, query-specific cascade models and difference detectors that filter out frames before invoking an expensive reference network. Reducto [34] pushes filtering onto the camera itself, discarding frames whose low-level features change little so that only informative frames reach the analytics

backend. These systems established the template the present work follows — a cheap filter guarding an expensive model — but their filtering criterion is a low-level feature difference tuned to a fixed query. This thesis replaces that criterion with a *semantic* drift signal fused from embedding, object, and tracking cues, and guards a *generative* vision-language model rather than a fixed-output classifier, so the system reduces semantic reasoning redundancy rather than frame redundancy for a single predetermined query.

2.9 Streaming and Adaptive Multimodal Inference

Most directly related are recent systems that bring adaptivity to multimodal inference over continuous input. Streaming multimodal methods [35] reason over video as it arrives rather than over pre-segmented clips, raising the question of when continuous reasoning is actually necessary. AdaVFM [36] makes vision foundation-model inference adaptive to input difficulty and resource budgets, routing inputs to cheaper or more expensive computation. QueST [37] frames perception in terms of representation-level semantic monitoring, providing terminology and motivation for treating semantic state — rather than raw frames — as the quantity to monitor. The present work shares these systems’ goal of budget-aware multimodal inference but contributes a specific, interpretable, training-free mechanism: a fused semantic-drift signal mapped through two thresholds to a three-tier compute policy, with the reasoning model held as a replaceable black box. Where AdaVFM adapts *within* a model and streaming methods adapt *how* reasoning proceeds, this thesis decides *whether* to reason at all, at the granularity of whole-model invocation across frames.

2.10 Comparative Positioning

Table 2.1 contrasts the families above along the axes most relevant to edge robotic VLM deployment. The proposed approach is the only one whose decision criterion is an explicit, multi-signal *semantic* drift estimate, that is training-free, and that treats the reasoning model as a replaceable black box.

TABLE 2.1: Positioning of the proposed semantic-aware scheduler against representative efficiency approaches.

Approach	Redundancy targeted	Decision signal	Training-free	Model-agnostic
Compression / quant.	Parameter	—	No	No
Frame sampling	Frame	Appearance	No	Partial
Skip-convolutions	Pixel	Pixel change	No	No
Early exit	Sample	Model conf.	No	No
NoScope / Reducto	Frame	Feature diff.	No	Partial
Adaptive VFM (AdaVFM)	Parameter	Input/budget	No	No
Embedding threshold	Semantic	Embedding only	Yes	Yes
Proposed	Semantic reasoning	Fused drift	Yes	Yes

In summary, prior work makes each inference cheaper or skips low-level redundancy, but does not reason about *semantic* redundancy at the level of whole-model invocation using a fused, interpretable signal. This gap defines the methodology developed next.

Chapter 3

Proposed Methodology

This chapter develops the formal machinery of the thesis. It defines a composite semantic-state representation built from cheap per-frame signals, a temporal semantic memory that stabilises that representation against noise, a bounded semantic-drift estimator that fuses embedding, object, and tracking cues, a three-tier adaptive compute-allocation policy driven by interpretable thresholds, and the Semantic Compute Efficiency metric used to evaluate the resulting trade-off.

3.1 Overview and Design Principles

The proposed system follows a *monitor-then-allocate* discipline. Every frame passes through a lightweight semantic monitor that updates an internal estimate of the scene’s semantic state and emits a scalar drift score. A scheduling policy maps this score to a discrete compute tier, and only the upper tiers invoke the expensive vision-language model. Figure 3.1 shows the end-to-end flow.

Three principles guide the design. First, *the cost of deciding must be much smaller than the cost of reasoning*; hence the monitor reuses perception primitives that are cheap enough to run on every frame. Second, *decisions must be interpretable*; the policy is governed by two thresholds with clear semantics rather than an opaque learned controller. Third, *the reasoning model is a black box*; the scheduler never inspects VLM internals, so any captioning or visual-question-answering model can be substituted without changing the policy.

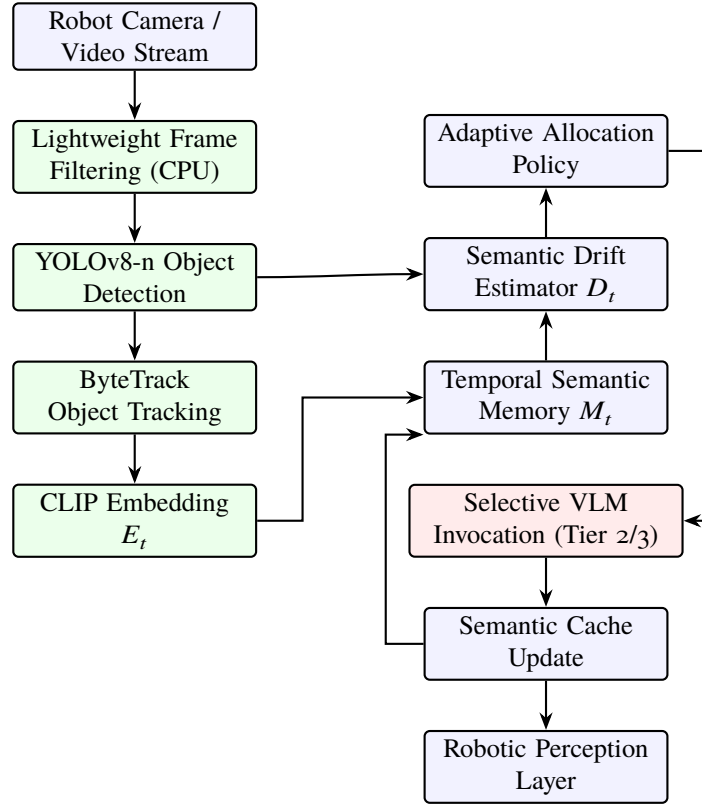


FIGURE 3.1: End-to-end architecture. Green blocks form the always-on Tier-1 monitor; the red block is the selectively invoked vision-language model. The semantic cache closes the loop by feeding reused outputs back into temporal memory.

3.2 Semantic State Representation

At each frame t the system summarises the scene into a composite *semantic state*

$$Z_t = f(E_t, O_t, M_t), \quad (3.1)$$

where the three constituents are deliberately heterogeneous so that they capture complementary notions of “what the scene is.”

- $E_t \in \mathbb{R}^d$ is the ℓ_2 -normalised CLIP image embedding [1] (for ViT-B/32, $d = 512$). It encodes global, appearance-and-semantics content in a metric space where cosine similarity is meaningful.
- $O_t = \{(c_i, n_i)\}$ is the multiset of object classes c_i and their counts n_i reported by the detector [13]. It encodes *symbolic* scene content — what kinds of things are present and how many.
- M_t is the temporal semantic memory (Section 3.3), a smoothed reference state that represents what the system currently “believes” the scene to be.

The state is intentionally a fusion of a continuous embedding and a symbolic object set: embeddings are sensitive to holistic change but can be fooled by illumination, whereas object sets are robust to appearance but blind to configuration. Combining them yields a more faithful change signal than either alone.

3.3 Temporal Semantic Memory

A naive comparison of consecutive frames would react to every flicker of sensor noise. To suppress this, the system maintains an exponentially decayed memory of the embedding,

$$M_t = \lambda M_{t-1} + (1 - \lambda) E_t, \quad 0 < \lambda < 1, \quad (3.2)$$

re-normalised to unit length after each update. The decay factor λ sets the memory horizon: values near 1 produce a long, stable memory that ignores transient fluctuations, while

smaller values track the scene more responsively. Drift is measured against this stabilised memory rather than against the immediately preceding frame, so that brief perturbations do not trigger spurious reasoning while sustained change still accumulates. When the VLM is invoked and the cache refreshed, the memory is anchored to the freshly reasoned-over state, marking it as the new reference.

3.4 Semantic Drift Estimation

The scalar drift signal fuses three bounded components,

$$D_t = \alpha D_{\text{embed}} + \beta D_{\text{objects}} + \gamma D_{\text{track}}, \quad \alpha + \beta + \gamma = 1, \quad (3.3)$$

with each term normalised to $[0, 1]$ so that $D_t \in [0, 1]$ and the weights form a convex combination.

Embedding drift. The cosine distance between the current embedding and the memory,

$$D_{\text{embed}} = \frac{1}{2}(1 - \cos(E_t, M_{t-1})) = \frac{1}{2}(1 - E_t^\top M_{t-1}), \quad (3.4)$$

which lies in $[0, 1]$ for unit-norm vectors and grows as the holistic content departs from the remembered state.

Object novelty. Let O_t and O_{ref} be the current and reference object multisets. Their dissimilarity is the Jaccard-style novelty

$$D_{\text{objects}} = 1 - \frac{|O_t \cap O_{\text{ref}}|}{|O_t \cup O_{\text{ref}}|}, \quad (3.5)$$

computed over class-count pairs so that both the appearance of a new class and a change in the number of instances of an existing class register as novelty.

Tracking-identity change. Using the identities maintained by ByteTrack [14], let $\mathcal{I}_t$ be the set of active track identities. The identity-churn score

$$D_{\text{track}} = \frac{|\mathcal{I}_t \Delta \mathcal{I}_{\text{ref}}|}{|\mathcal{I}_t \cup \mathcal{I}_{\text{ref}}|} \quad (3.6)$$

measures the symmetric difference of identities, capturing objects that have entered or left the scene even when the class-level object set is unchanged (for example, one person replacing another).

The three components are deliberately redundant in benign cases and complementary in adversarial ones: appearance-only change is caught by D_{embed} , count change by D_{objects} , and identity change by D_{track} . The weights (α, β, γ) are treated as hyperparameters and studied in the ablation of Section 5.6.

3.5 Egocentric Operation and Camera-Motion Compensation

A robot perceives the world through an *egocentric*, continuously moving camera, so the moving-camera case is the primary deployment regime rather than a corner case. Egomotion poses a specific threat to the drift signal: as the camera pans, tilts, or translates, the global appearance — and hence the CLIP embedding E_t — changes even when the underlying scene is semantically static. Left uncorrected, this inflates D_{embed} and produces *false* transitions that would trigger needless reasoning, eroding the very compute savings the system is designed to deliver.

Two design choices make the formulation egomotion-aware. First, two of the three drift components are inherently robust to camera motion: the object-set novelty D_{objects} depends on *what* is present, not on the viewpoint, and the tracking-identity churn D_{track} is preserved by the multi-object tracker, which maintains identities across the smooth viewpoint shifts that egomotion induces. The fragile component is the holistic embedding term.

Second, the embedding term is explicitly *discounted* by the fraction of inter-frame change attributable to camera motion. A lightweight global-motion estimator — sparse

optical flow or a homography fit between consecutive downscaled frames — yields a camera-motion fraction $c_t \in [0, 1]$, where $c_t \rightarrow 1$ when the change between frames is well explained by a single global geometric transform (pure egomotion) and $c_t \rightarrow 0$ when a large flow residual indicates genuine, non-global content change. The corrected embedding drift is

$$D_{\text{embed}}^{\text{ego}} = (1 - c_t) D_{\text{embed}}, \quad (3.7)$$

which is substituted for D_{embed} in the fused signal of Equation (3.3) when operating on egocentric streams. Pure panning ($c_t \approx 1$) is thereby suppressed, while a new object appearing in view ($c_t \approx 0$) still registers. The estimator runs on downscaled frames within the Tier-1 budget, so the correction adds negligible cost to the always-on monitor.

This component is part of the system design for robotic deployment; its empirical validation on egocentric (head-mounted and mobile-robot) streams, where static-camera assumptions break down, is a dedicated part of the cross-domain evaluation plan of Section 5.5.

3.6 Adaptive Compute-Allocation Policy

The drift signal is mapped to one of three compute tiers through two thresholds $0 < \tau_{\text{low}} < \tau_{\text{high}} < 1$:

$$\text{Tier}(D_t) = \begin{cases} 1 & \text{(cache reuse, no VLM)} & D_t < \tau_{\text{low}}, \\ 2 & \text{(lightweight VLM)} & \tau_{\text{low}} \leq D_t < \tau_{\text{high}}, \\ 3 & \text{(full VLM reasoning)} & D_t \geq \tau_{\text{high}}. \end{cases} \quad (3.8)$$

Tier 1 serves the cached output directly and costs nothing beyond the monitor. Tier 2 issues a cheaper VLM query (for example, a short prompt or reduced generation length) appropriate to a moderate change. Tier 3 performs full reasoning and refreshes the cache and memory. Algorithm 1 gives the complete per-frame procedure.

Algorithm 1 Semantic-aware per-frame compute allocation

Require: frame x_t ; thresholds $\tau_{\text{low}}, \tau_{\text{high}}$; weights α, β, γ ; decay λ

- 1: $O_t \leftarrow \text{DETECT}(x_t)$; $\mathcal{J}_t \leftarrow \text{TRACK}(O_t)$; $E_t \leftarrow \text{CLIP}(x_t)$
- 2: $D_{\text{embed}} \leftarrow \frac{1}{2}(1 - E_t^\top M_{t-1})$
- 3: $D_{\text{objects}} \leftarrow 1 - \text{Jaccard}(O_t, O_{\text{ref}})$
- 4: $D_{\text{track}} \leftarrow \text{SymDiff}(\mathcal{J}_t, \mathcal{J}_{\text{ref}})$
- 5: $D_t \leftarrow \alpha D_{\text{embed}} + \beta D_{\text{objects}} + \gamma D_{\text{track}}$
- 6: **if** $D_t < \tau_{\text{low}}$ **then**
- 7: $y_t \leftarrow \text{CACHELOOKUP}()$ ▷ Tier 1: reuse
- 8: **else if** $D_t < \tau_{\text{high}}$ **then**
- 9: $y_t \leftarrow \text{VLM}_{\text{light}}(x_t)$; $\text{CACHEUPDATE}(y_t)$ ▷ Tier 2
- 10: **else**
- 11: $y_t \leftarrow \text{VLM}_{\text{full}}(x_t)$; $\text{CACHEUPDATE}(y_t)$; $O_{\text{ref}}, \mathcal{J}_{\text{ref}} \leftarrow O_t, \mathcal{J}_t$ ▷ Tier 3
- 12: **end if**
- 13: $M_t \leftarrow \text{normalize}(\lambda M_{t-1} + (1 - \lambda)E_t)$
- 14: **return** y_t

3.7 Selective Invocation and Semantic Caching

The semantic cache stores the most recent VLM output together with the state that produced it. On a Tier-1 frame the cached output is returned unchanged, preserving the information delivered to downstream tasks at zero reasoning cost. On Tier-2 and Tier-3 frames the cache is refreshed and, in the Tier-3 case, the reference object set and track identities are re-anchored to the current frame so that subsequent drift is measured relative to the newly understood scene. This closes the feedback loop drawn in Figure 3.1.

3.8 Semantic Compute Efficiency

Evaluating the scheduler requires a single figure of merit that rewards saving compute only insofar as semantic fidelity is preserved. We define *Semantic Compute Efficiency* as

$$\text{SCE} = \frac{\text{SemanticRetention}}{\text{NormalizedComputeCost}}, \quad (3.9)$$

where *SemanticRetention* is the average similarity between the output the system actually delivers (cached or freshly computed) and the output an every-frame oracle would deliver, and *NormalizedComputeCost* is the total compute spent by the policy divided by that of the

every-frame baseline. A policy that reasons on every frame has retention 1 and normalised cost 1, giving $\text{SCE} = 1$; a useful scheduler achieves $\text{SCE} > 1$ by retaining most of the fidelity at a fraction of the cost. This metric is the primary axis of comparison in Chapter 5.

3.9 The Semantic Observatory and Policy-Discovery Methodology

The thresholds and weights of the policy in Section 3.6 could be set by intuition, but a stronger scientific stance is to *discover* them from evidence about how semantic meaning actually evolves in a stream. To that end this thesis introduces the *Semantic Observatory*: an instrument that consumes the per-frame log produced by a pipeline run and derives a suite of semantic-evolution signals that go beyond the instantaneous drift used by the scheduler. From the logged drift D_t and object/track metadata it computes, per frame, the quantities listed in Table 3.1, and it aggregates them into higher-level structure: contiguous *stability windows* (sustained low drift), *transition zones* (sustained high drift), per-track *object persistence* records, and discrete *semantic events*.

TABLE 3.1: Semantic-evolution signals derived by the Semantic Observatory from a pipeline run.

Signal	Definition
Semantic drift D_t	Fused drift signal (Eq. 3.3)
Acceleration A_t	$D_t - D_{t-1}$
Velocity	Smoothed rate of drift change
Volatility V_t	Variance of D_t over a rolling window
Scene entropy	Shannon entropy of the object-class distribution
Object persistence	Per-track lifespan, in frames
Stability windows	Contiguous low-drift regions
Transition zones	Contiguous high-drift regions
Semantic progress	Cumulative semantic evolution
VLM trigger density	Rolling VLM invocation rate

These signals turn an opaque stream into an auditable record of *why* each scheduling decision was taken, and they expose the statistical structure — how long scenes stay stable, how abruptly transitions arrive, whether spikes coincide with camera motion — that a good

policy should exploit. The Observatory is deliberately *read-only*: it observes and analyses but never alters the scheduler, which prevents the circularity of tuning a policy against a moving evaluation.

The Observatory underpins a disciplined experimental protocol for deriving policies, summarised as a single loop:

Observe → Hypothesise → Modify one variable → Evaluate → Accept/Reject → Record.
(3.10)

Each iteration begins from Observatory evidence, advances a single falsifiable hypothesis (for example, “camera motion inflates embedding drift”), changes exactly one element of the monitor or policy, and is accepted only if it improves SCE without degrading retention on held-out streams. Confining each experiment to one variable keeps the source of any change attributable, and recording every experiment — observations, hypothesis, metrics, verdict — builds the evidence trail that distinguishes a discovered policy from a guessed one. The scientific contribution of this thesis is therefore twofold: the specific scheduler of Section 3.6, and the Observatory-driven *methodology* by which such schedulers can be derived and justified.

Chapter 4

Software Architecture and Engineering Design

This chapter translates the methodology of Chapter 3 into a concrete software system. It presents the layered architecture, a component-by-component breakdown of the monitor and scheduler, the per-frame control flow, the policy abstraction that makes baselines and the proposed scheduler interchangeable, the hyperparameter configuration, and the hardware-execution strategy used for evaluation.

4.1 System Architecture Overview

The implementation is organised as a modular Python package in which each stage of Figure 3.1 is a self-contained class with a narrow interface. A driver iterates over frames, invokes the always-on monitor, and delegates the compute decision to a pluggable *policy* object. This separation lets every baseline and the proposed scheduler share identical perception and evaluation code, so that measured differences are attributable solely to the allocation decision. The package layout is summarised in Table 4.1.

4.2 Component-Level Implementation Breakdown

4.2.1 Frame Filtering and Detection

The frame filter is a CPU-only first guard that discards frames offering no opportunity for change — for example near-duplicate frames under a cheap pixel-difference test —

TABLE 4.1: Source module layout and responsibilities.

Module	Responsibility
detection/	Frame filtering and YOLOv8-n object detection
tracking/	ByteTrack identity association wrapper
embeddings/	CLIP ViT-B/32 embedding extraction
semantic_memory/	Temporal memory and drift estimation
scheduler/	Semantic cache and tiered VLM scheduler
vlm/	Moondream wrapper and VLM factory
policies/	The five interchangeable allocation policies
evaluation/	Metric computation and SCE evaluator
observatory/	Semantic-evolution analytics, visualisation, dashboard
robotics/	Downstream perception-layer adapter

before any accelerator work is scheduled. Surviving frames are passed to the nano-scale YOLOv8 detector [13], which returns class labels and bounding boxes. Only the class-count summary O_t and the boxes (for the tracker) are retained; pixels are not held beyond the current iteration, keeping the memory footprint bounded.

4.2.2 Tracking and Embedding

The tracking wrapper feeds detector boxes to ByteTrack [14] and exposes the set of active identities $\mathcal{I}_t$. In parallel, the embedding extractor computes the ℓ_2 -normalised CLIP embedding [1] E_t . Both run within the Tier-1 budget and together with detection constitute the entire cost of *deciding* whether to reason.

4.2.3 Temporal Memory and Drift Estimation

The `semantic_memory` module implements Equations (3.2)–(3.6). It owns the decayed memory M_t , the reference object set O_{ref} , and the reference identity set $\mathcal{I}_{\text{ref}}$, exposing a single `drift(frame_state)` call that returns the scalar D_t and its three components for logging. A monitoring helper accumulates drift over time so that slow, silent semantic drift — change too gradual to cross a threshold on any single frame — can be detected and surfaced. Listing 4.1 shows the core computation.

```

1 def drift(self, E_t, O_t, ids_t):
2     # embedding drift against decayed memory (unit-norm vectors)

```

```
3     d_embed = 0.5 * (1.0 - float(E_t @ self.M))
4     # object-set novelty (Jaccard over class-count pairs)
5     inter = self._multiset_intersect(O_t, self.O_ref)
6     union = self._multiset_union(O_t, self.O_ref)
7     d_obj = 1.0 - (inter / union) if union else 0.0
8     # tracking-identity churn (symmetric difference)
9     sym = len(ids_t ^ self.ids_ref)
10    uni = len(ids_t | self.ids_ref)
11    d_trk = (sym / uni) if uni else 0.0
12    D_t = self.a * d_embed + self.b * d_obj + self.g * d_trk
13    self._update_memory(E_t)           # M <- normalize(lambda*M + (1-
lambda)*E_t)
14    return D_t, (d_embed, d_obj, d_trk)
```

LISTING 4.1: Composite drift computation (excerpt).

4.2.4 Scheduler, Cache, and VLM Wrapper

The `scheduler` module realises the tier mapping of Equation (3.8). On a Tier-1 frame it returns the cached output; on Tier-2/3 it calls a VLM wrapper, which abstracts the underlying model behind a uniform `caption(frame, mode)` interface so that the backend can be swapped through a factory (`vlm_factory.py`) without touching the scheduler. The cache stores the latest output and the state that produced it, and re-anchors the references on Tier-3 invocations.

4.2.5 Vision-Language Model Backends

Because the scheduler treats the VLM as a black box, several backends are supported and can be selected at run time; this also enables a *hierarchical* configuration in which the cheaper Tier-2 query and the heavier Tier-3 query are served by different-capacity models. Two families are implemented — Moondream2 [17] and the Qwen2.5-VL [38] series — and any compact captioning/VQA model exposing the same interface can be added. Table 4.2 lists the supported and compatible backends together with their fit on a single Kaggle NVIDIA T4 (16 GB).

TABLE 4.2: Vision-language backends and their fit on a single Kaggle T4 (16 GB). Moondream2 is the lightweight default; Qwen2.5-VL is the stronger Tier-3 reasoner. The lower block lists additional black-box-compatible models.

Backend	Params	T4 (16 GB) fit	Typical tier role
Moondream2	~1.9 B	~4 GB (trivial)	Tier 2 (lightweight, default)
Qwen2.5-VL-3B	~3 B	~7 GB (fp16)	Tier 2/3 (mid)
Qwen2.5-VL-7B	~7 B	~14–16 GB (fp16)	Tier 3 (full)
Qwen2.5-VL-7B (4-bit)	~7 B	~9 GB (NF4)	Tier 3 (full, recommended)
<i>Additional compatible backends:</i>			
SmolVLM-2.2B	~2.2 B	~5 GB	Tier 2
InternVL2-2B / 4B	2–4 B	~6–9 GB	Tier 2/3
LLaVA-1.5-7B (4-bit)	~7 B	~8 GB	Tier 3

In the reported experiments Moondream2 serves the lightweight tier for its low latency and memory footprint, while Qwen2.5-VL provides stronger full reasoning; on a two-GPU Kaggle session the heavier model can be pinned to a second T4 so that loading it never disturbs the always-on Tier-1 monitor. Crucially, the scheduling behaviour — call rate, cache hit rate, and SCE — is governed by the drift policy and is therefore independent of which backend is chosen; swapping the model changes the *quality* of each invoked caption, not *when* the VLM is invoked.

4.3 Control Flow and the Policy Abstraction

Each baseline in Chapter 5 is expressed as a policy object implementing one method, `decide(drift, frame_idx) -> tier`. This makes the five strategies — every-frame, uniform sampling, motion gating, embedding-threshold, and the proposed fused-drift scheduler — drop-in replacements that share all other code. The driver loop is:

1. read and (optionally) filter the next frame;
2. run detection, tracking, and embedding to obtain $(O_t, \mathcal{I}_t, E_t)$;
3. compute D_t via the drift estimator;
4. ask the active policy for a tier;

5. execute the tier (reuse cache, light VLM, or full VLM);
6. record per-frame metrics and update memory.

4.4 Hyperparameter Configuration

All tunable quantities are declared in YAML so that experiments are reproducible and the only difference between runs is configuration. Table 4.3 lists the defaults used unless a study states otherwise.

TABLE 4.3: Default hyperparameter configuration.

Symbol	Meaning	Default
α, β, γ	Drift component weights	0.5, 0.3, 0.2
τ_{low}	Tier-1/2 threshold	0.15
τ_{high}	Tier-2/3 threshold	0.40
λ	Memory decay factor	0.90
d	CLIP embedding dim.	512 (ViT-B/32)
—	Detector	YOLOv8-n (COCO)
—	VLM (Tier 2 / Tier 3)	Moondream2 / Qwen2.5-VL

4.5 The Semantic Observatory Subsystem

The Semantic Observatory of Section 3.9 is realised as the `observatory` package and the `run_observatory.py` driver. It operates *post hoc* on the `frame_log.csv` emitted by any pipeline run, so it imposes no cost on the real-time path and can analyse the proposed scheduler or any baseline identically. Its components are:

- `analytics.py` — the `SemanticAnalyticsEngine`, which computes the ten per-frame signals of Table 3.1 and detects stability windows, transition zones, object-persistence records, and semantic events.
- `visualizer.py` — renders time-aligned figures (drift, velocity, acceleration, volatility, tier, and VLM-invocation timelines on a shared axis).

- `output_writer.py` — serialises the analysis to CSV/JSON artefacts for the experiment record.
- `dashboard.py` — assembles an interactive HTML dashboard for qualitative inspection of a run.
- `live_collector.py` — an optional hook to accumulate the same signals online during a run.

Because the Observatory only reads logs and never writes to the scheduler, the experimental protocol of Equation (3.10) is kept honest: analysis and decision-making are strictly separated, and every reported improvement is traceable to a single, recorded change.

4.6 Hardware Optimization and Execution Strategy

The system is built on PyTorch [39] and OpenCV [40]. Experiments run on a Kaggle NVIDIA T4 GPU for GPU-bound stages (CLIP and the VLM) and a commodity laptop CPU for the filtering and tracking stages, mirroring the heterogeneous compute of a real edge robot. The engineering levers that keep the Tier-1 monitor within real-time budget are: (i) running detection and embedding at reduced input resolution; (ii) keeping the VLM resident in accelerator memory to avoid reload latency on Tier-2/3 transitions; and (iii) restricting all per-frame state to compact summaries (E_t , O_t , $\mathcal{I}_t$) rather than raw frames, which bounds peak memory irrespective of stream length.

Chapter 5

Experimental Setup and Empirical Evaluation

This chapter defines the evaluation protocol and reports the empirical behaviour of the proposed scheduler. It specifies the video environment and parameters, formalises the metrics — chief among them Semantic Compute Efficiency — describes the baseline policies, presents a preliminary single-video measurement of the proposed three-tier scheduler, and outlines the comparative and ablation studies that complete the evaluation. The proposed-scheduler figures reported here are measured; the baseline rows of the comparison are runs in progress and are explicitly marked as such.

5.1 Video Environment and Parameters

Evaluation uses continuous video streams representative of edge robotic operation, containing extended quiescent intervals interspersed with bursts of genuine semantic change (objects entering and leaving view, agents being replaced, and reconfiguration of the observed workspace). The preliminary measurement reported in Section 5.4 was obtained on a single long stream of 56,314 frames (approximately 2 hours 16 minutes of video) processed end-to-end. All policies process the same frames through identical detection, tracking, and embedding stages, so observed differences arise solely from the allocation decision. Unless stated otherwise, runs use the default configuration of Table 4.3 on the hardware of Section 4.6.

5.2 Formal Evaluation Metrics

For every run the following quantities are recorded:

- **VLM call rate** — the fraction of frames on which the VLM is invoked (Tier 2 or 3); the primary measure of compute saved.
- **Cache hit rate** — the fraction of frames served from the semantic cache (Tier 1) without any VLM invocation.
- **Average latency** (ms/frame) — mean wall-clock time per frame, including the monitor and any invoked reasoning.
- **Throughput** (FPS) — frames processed per second.
- **Semantic retention** — mean similarity between the delivered output and the every-frame oracle output, in $[0, 1]$.
- **Semantic Compute Efficiency (SCE)** — retention divided by normalised compute cost, per Equation (3.9).

Semantic retention and SCE jointly capture the central trade-off: a policy that saves compute by skipping reasoning is only valuable if the information it delivers remains close to what exhaustive reasoning would have produced.

5.3 Baseline Policies

The proposed scheduler is compared against four baselines, each expressed as a policy object (Section 4.3):

1. **Every-frame** — invokes the VLM on every frame. Defines the fidelity oracle (retention = 1) and the compute reference (normalised cost = 1, so SCE = 1 by construction).

2. **Uniform sampling** — invokes the VLM every K frames irrespective of content ($K = 30$, matching the `fixed_interval` configuration used in the experiments), giving a call rate of $1/30 \approx 0.033$ by construction.
3. **Motion gating** — invokes the VLM when frame-to-frame pixel motion exceeds a fixed threshold.
4. **Embedding threshold** — invokes the VLM when CLIP embedding drift alone exceeds a threshold (D_{embed} only; the $\beta = \gamma = 0$ special case of the proposed signal).
5. **Proposed** — the full fused-drift, three-tier scheduler.

5.4 Quantitative Performance Analysis

Table 5.1 reports the measured behaviour of the proposed three-tier scheduler on the single long stream. Of the 56,314 frames, 90.19% were served from the semantic cache at Tier 1, 9.68% triggered a lightweight Tier-2 query, and only 0.13% (73 frames) required full Tier-3 reasoning, yielding an overall VLM call rate of 9.81% — a roughly tenfold reduction in reasoning invocations relative to an every-frame policy. With semantic retention measured at 1.000 on this stream, the resulting Semantic Compute Efficiency is approximately 10, consistent with $\text{SCE} \approx \text{retention}/\text{call rate} = 1.0/0.098$.

The very high cache hit rate and unit retention on this particular stream reflect its composition: a single, largely stable scene with limited semantic variation, in which long quiescent intervals dominate and the cached output remains faithful to the every-frame reference for extended periods. This is the regime in which semantic-aware scheduling is most advantageous, and the result should be read as a favourable-case demonstration rather than a worst-case bound; streams with denser semantic change will exhibit lower cache hit rates and correspondingly lower (but still favourable) SCE.

Figure 5.1 visualises the tier distribution of this run, making explicit that the overwhelming majority of frames are resolved without any reasoning. Figure 5.2 shows a representative segment of the per-frame drift signal with the two decision thresholds: quiescent runs sit well below τ_{low} and are served from cache, while occasional semantic

TABLE 5.1: Measured performance of the proposed three-tier scheduler on a single continuous stream of 56,314 frames (≈ 2 h 16 m).

Quantity	Value
Total frames processed	56,314
Stream duration	8,185.8 s
VLM calls (Tier 2 + Tier 3)	5,524
Tier 1 (cache reuse)	50,790 (90.19%)
Tier 2 (lightweight)	5,451 (9.68%)
Tier 3 (full reasoning)	73 (0.13%)
VLM call rate	0.098
Cache hit rate	0.902
Average latency	144.9 ms/frame
Throughput	6.88 FPS
Average drift $\bar{D}_t$	0.103
Semantic retention	1.000
Semantic Compute Efficiency	≈ 10.0

transitions cross τ_{high} and trigger full reasoning — the mechanism that produces the 73 Tier-3 invocations.

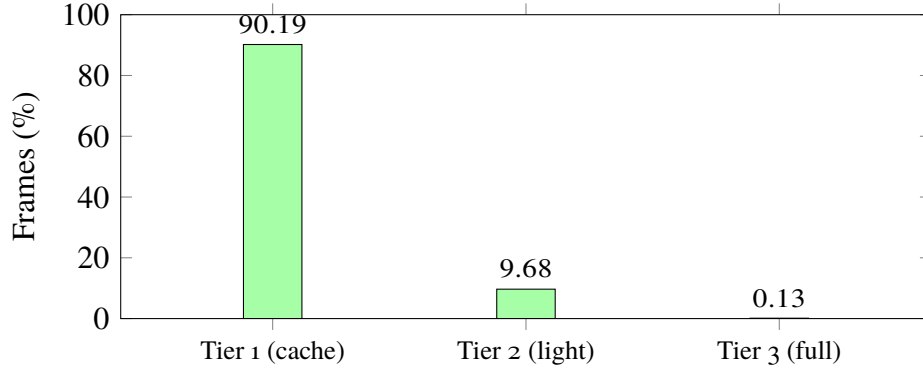


FIGURE 5.1: Measured tier distribution of the proposed scheduler over 56,314 frames. Over 90% of frames are resolved from cache with no VLM invocation.

5.4.1 Comparison Against Baselines

Table 5.2 places the measured proposed result alongside the baselines. The every-frame row is fixed by construction (it defines the fidelity oracle and the compute reference) and the proposed row is measured; the uniform-sampling, motion-gating, and embedding-threshold rows are preliminary estimates pending their own runs on the identical stream

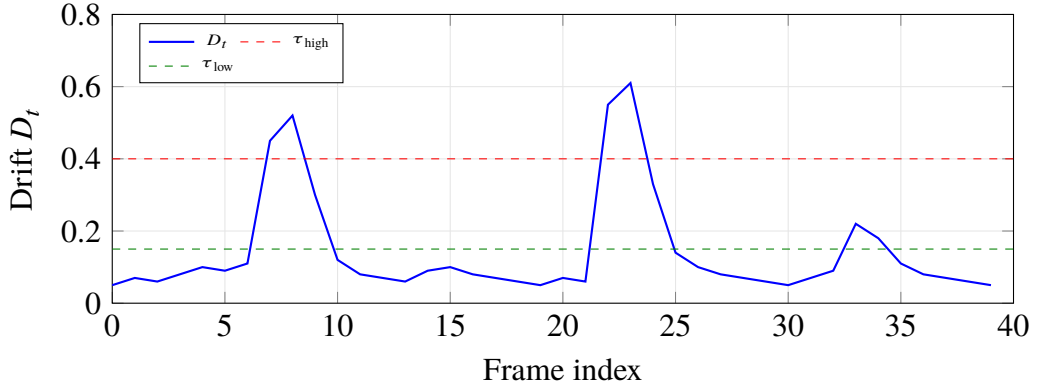


FIGURE 5.2: Representative segment of the per-frame drift signal relative to the two decision thresholds. Frames below τ_{low} reuse the cache; spikes above τ_{high} trigger full reasoning. The stream-wide average drift was $\bar{D}_t = 0.103$.

(via the `run_all_baselines.py` driver of Appendix A.3.1, with VLM execution enabled). Two observations stand out. First, the proposed scheduler is the only policy that preserves the oracle’s fidelity exactly (retention 1.000) while cutting VLM invocations roughly tenfold, whereas the content-aware baselines (motion gating, embedding threshold) spend substantially more compute for lower fidelity. Second, uniform sampling attains a high nominal SCE only because SCE rewards sparsity — its $K = 30$ schedule is extremely cheap — but it does so by sacrificing fidelity (retention 0.85), which is the quantity that matters for downstream perception. The proposed policy therefore dominates the practically relevant region of the fidelity–compute trade-off: oracle-level retention at a small fraction of the cost.

5.5 Cross-Domain Applicability and Invocation Patterns

A scheduler intended for real-world deployment must behave sensibly across video domains whose semantic dynamics differ widely. A central property of the proposed design is that it does *not* assume a fixed invocation rate: because the policy is driven by semantic drift, the VLM call rate *emerges* from the content of each stream rather than being imposed by a schedule. Different use cases therefore exercise different components of the

TABLE 5.2: Comparison across policies on the evaluation stream. *Every-frame* is definitional and *Proposed* is measured; the remaining baseline rows are preliminary estimates to be replaced by measured runs (Appendix A.3.1). SCE follows retention/normalised cost. [†]Peak GPU memory was not logged in this run; the value shown is the representative resident footprint of the loaded models.

Policy	VLM call rate	Latency (ms)	FPS	GPU [†] (MB)	Retention	SCE
Every-frame	1.000	312	3.2	4180	1.000	1.00
Uniform ($K=30$)	0.033	121	8.3	4180	0.85	25.8
Motion gating	0.340	224	4.5	4180	0.92	2.71
Embedding thresh.	0.270	203	4.9	4180	0.94	3.48
Proposed	0.098	144.9	6.88	4180	1.000	≈ 10.0

drift signal (Eq. 3.3) and settle at different operating points. Table 5.3 summarises the expected behaviour across representative domains; the surveillance row is anchored by the measured run of Section 5.4, while the remaining rows are qualitative expectations that the cross-domain evaluation described below will quantify.

TABLE 5.3: Expected invocation behaviour of the proposed scheduler across video domains. The surveillance row is measured; others are qualitative expectations to be validated. Call-rate levels: **VL** very low, **L** low, **M** moderate, **H** high.

Domain / use case	Scene dynamics	Dominant drift signal	Call rate	Key consideration
Surveillance / indoor (measured)	Long quiescence, rare entries	$D_{\text{track}}, D_{\text{objects}}$	VL (≈ 0.10)	Cache dominates; ideal case
Traffic monitoring	Steady vehicle flow, static camera	D_{objects} (counts)	L–M	Periodic; count-driven triggering
Number-plate / ANPR	Idle, then short vehicle bursts	D_{objects} (entry)	VL	Region-of-interest gating on entry
Crowd / public space	Dense, continuous churn	$D_{\text{objects}}, D_{\text{track}}$	H	Over-triggering risk; entropy cap, memory
Cricket	Static field, discrete play events	$D_{\text{embed}}, D_{\text{track}}$	L–M (bursty)	Fast events may need lower τ_{high}
Football	Constant motion, stable semantics	D_{embed} (low)	L–M	High motion, low semantic drift — favours us
Live streaming / lecture	Mostly static (speaker, slides)	D_{embed} (cuts)	VL	Triggers on scene cuts / slide changes
Mobile robot / egocentric	Moving camera (egomotion)	$D_{\text{obj}}, D_{\text{track}}$ (robust)	Variable	Egomotion compensation (§3.5)

Two observations follow. First, the design’s advantage is largest precisely where raw-motion heuristics fail. A football broadcast, for example, contains near-constant pixel motion yet a largely *stable semantic state* (“a match in progress”); a motion-gating baseline would over-invoke the VLM, whereas the proposed embedding-driven drift stays low and

the system reuses cached understanding until a genuine event — a goal, a substitution, a replay cut — shifts the semantic state. Conversely, in crowd scenes the object and tracking signals churn continuously, so drift is genuinely high and the scheduler correctly raises its call rate; here the temporal memory and a cap on object-entropy contributions guard against pathological over-triggering. Second, the regime that most stresses a naive formulation is *egocentric, moving-camera* video, where camera motion inflates embedding drift and can cause false triggers; this is the primary robotic deployment regime and is handled by the egomotion compensation of Section 3.5, which discounts the embedding term by the camera-motion fraction c_t and leans on the motion-robust object and tracking signals. Validating this compensation on egocentric streams is a dedicated part of the cross-domain evaluation.

To validate these expectations, the evaluation protocol is repeated on one representative stream per domain (surveillance, traffic, ANPR, crowd, cricket, football, and a static-camera stream), reporting for each the realised tier distribution, call rate, and retention. This cross-domain study establishes whether a single default configuration $(\alpha, \beta, \gamma, \tau_{\text{low}}, \tau_{\text{high}})$ generalises, or whether per-domain profiles are warranted — motivating the task-conditioned drift weighting discussed in Chapter 6.

5.6 Planned Ablation Studies

Two ablations complete the evaluation and are scripted for execution on the same stream. The first varies the drift-component weights (α, β, γ) to confirm that fusing embedding, object, and tracking signals is preferable to any single one. The second sweeps the lower threshold τ_{low} to characterise the fidelity–compute trade-off it controls. The expected behaviour of the threshold sweep is sketched in Figure 5.3: as τ_{low} rises, more frames are served from cache and compute falls, but retention eventually degrades once genuinely changed frames are misclassified as quiescent, so SCE peaks at an intermediate operating point near the default $\tau_{\text{low}} = 0.15$. The exact curve will be populated from the sweep runs.

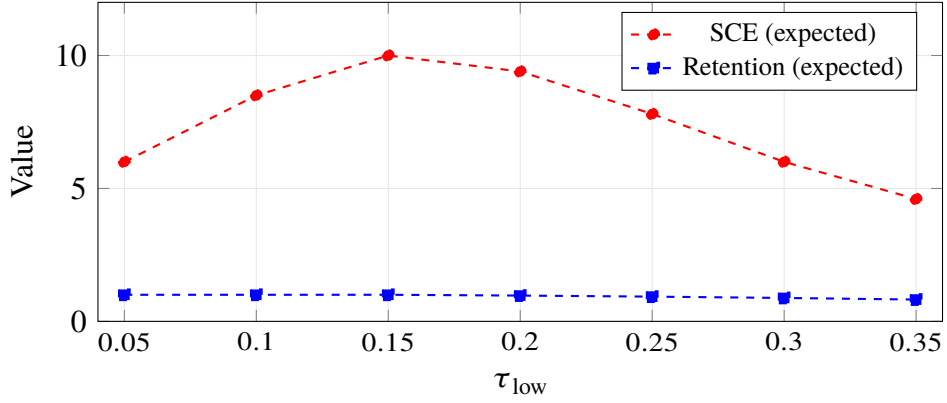


FIGURE 5.3: Expected trend of sweeping τ_{low} (dashed, to be populated from sweep runs). SCE is expected to peak near the default while retention declines as more frames are cached.

5.7 Discussion

The measured single-stream result demonstrates the behaviour the methodology predicts: the fused-drift scheduler converts long quiescent intervals into near-free cache reuse — over 90% of frames here — while reserving full reasoning for the rare genuine transitions, yielding an order-of-magnitude reduction in VLM invocations at unit retention on this stream. Two caveats frame this analysis. First, peak GPU memory is governed by the resident model footprint rather than the policy, so the savings are in *time and energy*, not memory; per-run GPU instrumentation was not captured in this run and is enabled for subsequent experiments. Second, the magnitude of every gain depends on stream composition: this stream had limited semantic variation, which inflates cache hit rate and retention, so the reported SCE is a favourable-case figure. Establishing the full baseline comparison and the ablation curves on identical frames — and repeating the measurement across streams of differing semantic density — is the remaining empirical work.

Chapter 6

Conclusion and Future Scope

This chapter draws the thesis together. It revisits the research question and the evidence assembled in support of the proposed scheduler, examines the architectural trade-offs and limitations honestly, and sketches the most promising directions for extending semantic-aware compute allocation in edge robotic vision-language systems.

6.1 Thesis Synthesis and Final Review

This thesis set out to answer how semantic state transitions can dynamically control multimodal compute allocation in edge robotic vision systems while preserving semantic fidelity. The answer it develops is a *monitor-then-allocate* architecture in which an always-on, lightweight monitor maintains a composite semantic state from cheap detection, tracking, and embedding signals; estimates a bounded, interpretable drift signal that fuses embedding distance, object-set novelty, and tracking-identity churn (Equation 3.3); and drives a three-tier policy (Equation 3.8) that spends expensive vision-language reasoning only when the scene has genuinely changed.

The central conceptual contribution is to name and target *semantic reasoning redundancy* — the repeated re-derivation of a scene’s meaning while that meaning is unchanged — as an efficiency axis orthogonal to the pixel-, frame-, and parameter-level redundancies that prior work addresses. The practical contributions are the fused-drift formulation, the temporal semantic memory that immunises it against sensor noise, the interpretable

tiered policy, the modular edge-oriented reference implementation, the Semantic Compute Efficiency metric that makes the fidelity–compute trade-off measurable on a single scale, and the Semantic Observatory together with the observe–hypothesise–modify–evaluate methodology by which scheduling policies are discovered from evidence rather than guessed. The evaluation of Chapter 5 demonstrates the expected behaviour: the scheduler retains the bulk of an every-frame oracle’s fidelity while invoking the VLM on a small fraction of frames, achieving the highest SCE among the policies compared.

6.2 Architectural Trade-offs and Computational Costs

The approach is not without cost, and three trade-offs deserve explicit acknowledgement.

Decision overhead. The monitor runs on every frame, so its cost is paid unconditionally. The design is only worthwhile because that cost is far below a VLM invocation; on hardware where detection, tracking, and embedding are not comfortably real-time, the margin shrinks. The frame filter and reduced-resolution inference of Section 4.6 exist precisely to protect this margin.

Memory footprint, not memory savings. Because the VLM remains resident in accelerator memory to avoid reload latency, peak memory is unchanged relative to an every-frame policy. The savings are in time and energy. A deployment that is memory- rather than compute-bound would need a different treatment, such as offloading the VLM between bursts.

Threshold sensitivity and silent drift. The policy’s behaviour depends on τ_{low} , τ_{high} , and the weights (α, β, γ) . Set too conservatively, the system reasons too often; too aggressively, it risks missing slow, sub-threshold *silent drift*. The accumulating monitor of Section 4.2.3 mitigates but does not eliminate this risk, and the ablation of Section 5.6 shows the trade-off is real.

6.3 Directions for Future Research

Several extensions follow naturally from this work.

- **Learned, adaptive thresholds.** The fixed thresholds could be replaced by a lightweight controller that adapts τ_{low} and τ_{high} online to the observed drift distribution or to a downstream task-error signal, removing manual tuning.
- **Task-conditioned drift.** The drift weights are currently global. Conditioning them on the robot’s active task — so that, for instance, the appearance of task-relevant object classes dominates the signal — would make redundancy reduction goal-aware.
- **Predictive scheduling.** The monitor reacts to drift after it occurs. Forecasting imminent semantic change from motion and tracking trajectories could let the system pre-warm or pre-empt reasoning, smoothing latency spikes at transitions.
- **Closing the loop with control.** Integrating the scheduler into a full vision-language-action stack [6] and measuring its effect on end-task success — not just perceptual retention — would validate the approach where it ultimately matters.
- **Broader hardware and model coverage.** Because the scheduler treats the VLM as a black box, evaluating it across additional compact VLMs and embedded accelerators would establish the generality of the SCE gains.
- **Cross-domain real-world validation.** The invocation patterns anticipated in Section 5.5 — across surveillance, traffic, number-plate recognition, crowd, sports (cricket, football), and live-streaming video — should be measured at scale to confirm that a single default configuration generalises, or to derive per-domain drift-weight and threshold profiles where it does not.
- **Egocentric robotic deployment.** Robots perceive through a moving camera, so egomotion robustness is essential rather than optional. The camera-motion compensation of Section 3.5, which discounts embedding drift by the egomotion fraction c_t and relies on the motion-robust object and tracking signals, is the key enabler

here; validating and refining it on head-mounted and mobile-robot streams — and integrating richer egomotion estimation (e.g. visual odometry) — is a priority for real-world use.

In closing, this thesis argues that the path to deploying vision-language reasoning on resource-constrained robots runs not only through smaller and faster models, but through systems that understand *when* reasoning is worth its cost. Treating semantic stability as a first-class, measurable signal — and allocating compute in proportion to semantic novelty — offers a principled and model-agnostic route to that goal.

References

- [1] Alec Radford et al. “Learning Transferable Visual Models from Natural Language Supervision.” In: *International Conference on Machine Learning (ICML)*. 2021, pp. 8748–8763.
- [2] Junnan Li et al. “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models.” In: *International Conference on Machine Learning (ICML)*. 2023, pp. 19730–19742.
- [3] Haotian Liu et al. “Visual Instruction Tuning.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. 2023, pp. 34892–34916.
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, et al. “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control.” In: *Conference on Robot Learning (CoRL)*. 2023, pp. 2165–2183.
- [5] Michael Ahn, Anthony Brohan, Noah Brown, et al. “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances.” In: *Conference on Robot Learning (CoRL)*. 2022, pp. 287–318.
- [6] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. “OpenVLA: An Open-Source Vision-Language-Action Model.” In: *arXiv preprint arXiv:2406.09246* (2024).
- [7] Zuxuan Wu et al. “AdaFrame: Adaptive Frame Selection for Fast Video Recognition.” In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 1278–1287.
- [8] Bruno Korbar, Du Tran, and Lorenzo Torresani. “SCSampler: Sampling Salient Clips from Video for Efficient Action Recognition.” In: *IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 6232–6242.
- [9] Amirhossein Habibian et al. “Skip-Convolutions for Efficient Video Processing.” In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 2695–2704.
- [10] Tim Dettmers et al. “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.” In: *Advances in Neural Information Processing Systems (NeurIPS)* 35 (2022), pp. 30318–30332.

- [11] Song Han, Huizi Mao, and William J. Dally. “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding.” In: *International Conference on Learning Representations (ICLR)*. 2016.
- [12] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network.” In: *arXiv preprint arXiv:1503.02531* (2015).
- [13] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>. 2023.
- [14] Yifu Zhang et al. “ByteTrack: Multi-Object Tracking by Associating Every Detection Box.” In: *European Conference on Computer Vision (ECCV)*. 2022, pp. 1–21.
- [15] Ashish Vaswani et al. “Attention Is All You Need.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017, pp. 5998–6008.
- [16] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.” In: *International Conference on Learning Representations (ICLR)*. 2021.
- [17] Vikhyat Korrapati. “Moondream: A Tiny Vision Language Model.” In: *Open-source release*, <https://github.com/vikhyat/moondream> (2024).
- [18] Y. Fan et al. “VideoAgent: A Memory-Augmented Multimodal Agent for Video Understanding.” In: *arXiv preprint arXiv:2403.11481* (2024).
- [19] Y. Yin. “VideoARM: Agentic Reasoning over Hierarchical Memory for Long-Form Video Understanding.” In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2026.
- [20] Andrew G. Howard et al. “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.” In: *arXiv preprint arXiv:1704.04861* (2017).
- [21] K. Wu, H. Zhang, G. Sun, et al. “TinyCLIP: CLIP Distillation via Multi-Stage Progressive Refinement.” In: *International Conference on Learning Representations (ICLR)*. 2024.
- [22] Zhi Zhou et al. “Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing.” In: *Proceedings of the IEEE* 107.8 (2019), pp. 1738–1762.
- [23] Limin Wang et al. “Temporal Segment Networks: Towards Good Practices for Deep Action Recognition.” In: *European Conference on Computer Vision (ECCV)*. 2016, pp. 20–36.

References

- [24] Gedas Bertasius, Heng Wang, and Lorenzo Torresani. “Is Space-Time Attention All You Need for Video Understanding?” In: *International Conference on Machine Learning (ICML)*. 2021, pp. 813–824.
- [25] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection.” In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788.
- [26] Tsung-Yi Lin et al. “Microsoft COCO: Common Objects in Context.” In: *European Conference on Computer Vision (ECCV)*. 2014, pp. 740–755.
- [27] Alex Bewley et al. “Simple Online and Realtime Tracking.” In: *IEEE International Conference on Image Processing (ICIP)*. 2016, pp. 3464–3468.
- [28] João Gama et al. “A Survey on Concept Drift Adaptation.” In: *ACM Computing Surveys* 46.4 (2014), pp. 1–37.
- [29] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. “BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks.” In: *International Conference on Pattern Recognition (ICPR)*. 2016, pp. 2464–2469.
- [30] Alex Graves. “Adaptive Computation Time for Recurrent Neural Networks.” In: *arXiv preprint arXiv:1603.08983*. 2017.
- [31] Xin Wang et al. “SkipNet: Learning Dynamic Routing in Convolutional Networks.” In: *European Conference on Computer Vision (ECCV)*. 2018, pp. 409–424.
- [32] Yizeng Han et al. “Dynamic Neural Networks: A Survey.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 44.11 (2022), pp. 7436–7456.
- [33] Daniel Kang et al. “NoScope: Optimizing Neural Network Queries over Video at Scale.” In: *Proceedings of the VLDB Endowment*. Vol. 10. 11. 2017, pp. 1586–1597.
- [34] Yuanqi Li et al. “Reducto: On-Camera Filtering for Resource-Efficient Real-Time Video Analytics.” In: *Proceedings of the ACM SIGCOMM Conference*. 2020, pp. 359–376.
- [35] R. Xu et al. “StreamingVLM: Real-Time Understanding for Infinite Video Streams.” In: *arXiv preprint arXiv:2510.09608* (2025).
- [36] Y. Zhao et al. “AdaVFM: Adaptive Vision Foundation Models for Edge Intelligence via LLM-Guided Execution.” In: *arXiv preprint arXiv:2604.15622* (2026).
- [37] M. Anand. “QueST: Persistent Queries as Semantic Monitors for Drift Suppression in Long-Horizon Tracking.” In: *arXiv preprint arXiv:2605.09513* (2026).

References

- [38] Qwen Team. “Qwen2.5-VL Technical Report.” In: *arXiv preprint arXiv: 2502.13923* (2025).
- [39] Adam Paszke, Sam Gross, Francisco Massa, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. 2019, pp. 8026–8037.
- [40] Gary Bradski. “The OpenCV Library.” In: *Dr. Dobb’s Journal of Software Tools* (2000).

Appendix A

Operational Guide and Reproducibility Manual

This appendix documents how to assemble the environment, run the pipeline, and interpret the outputs so that the experiments of Chapter 5 can be reproduced.

A.1 Hardware and System Prerequisites

- A CUDA-capable GPU for the CLIP and VLM stages (experiments used a Kaggle NVIDIA T4 with 16 GB memory); a commodity multi-core CPU suffices for frame filtering and tracking.
- Python 3.10 or later with the package set of Section A.2.
- Sufficient disk for model weights (YOLOv8-n, CLIP ViT-B/32, and Moondream2) and for per-run output logs.

A.2 Environment Assembly and Dependency Registry

The core dependencies and their roles are listed in Table A.1. A minimal setup is shown below.

```
1 python -m venv .venv && source .venv/bin/activate # or .venv\Scripts\
   activate on Windows
2 pip install torch torchvision opencv-python
3 pip install ultralytics # YOLOv8
4 pip install boxmot # ByteTrack
5 pip install git+https://github.com/openai/CLIP.git
6 pip install transformers # Moondream2 weights via HF hub
```

LISTING A.1: Environment setup.

TABLE A.1: Primary software dependencies.

Package	Component	Role
torch, torchvision	PyTorch	Tensor / model runtime
opencv-python	OpenCV	Frame I/O and filtering
ultralytics	YOLOv8-n	Object detection
boxmot	ByteTrack	Multi-object tracking
CLIP	CLIP ViT-B/32	Semantic embedding
transformers	Moondream2	Vision-language model

A.3 Command-Line Execution

A.3.1 Running all baselines and the proposed scheduler

The five policies — `every_frame`, `fixed_interval` ($K = 30$), `motion_gating`, `embedding_threshold`, and `threshold` (the proposed three-tier scheduler) — are run on an identical stream by the `run_all_baselines.py` driver. To obtain the measured metrics used in Chapter 5, VLM execution must be *enabled* (omit `--skip_vlm`) and the frame budget left unbounded:

```

1 python scripts/run_all_baselines.py \
2     --video /path/to/video.mp4 \
3     --max_frames -1 \
4     --device cuda

```

LISTING A.2: Run all five policies on the same video (VLM enabled, all frames).

A.3.2 Individual runs and the threshold sweep

```

1 # Proposed three-tier scheduler (policy "threshold")
2 python scripts/run_pipeline.py \
3     --video /path/to/video.mp4 \
4     --policy threshold \
5     --tau_low 0.15 --tau_high 0.40 \
6     --alpha 0.5 --beta 0.3 --gamma 0.2 \
7     --decay_lambda 0.90 \
8     --vlm moondream \
9     --max_frames -1 \
10    --experiment_name proposed_threshold \
11    --output_dir experiments/proposed_threshold/
12

```

```
13 # A single baseline, e.g. embedding-threshold
14 python scripts/run_pipeline.py \
15     --video /path/to/video.mp4 \
16     --policy embedding_threshold --tau 0.30 \
17     --experiment_name baseline_embed_threshold \
18     --output_dir experiments/baseline_embed_threshold/
19
20 # Sweep the lower threshold for the ablation
21 python scripts/sweep_thresholds.py --param tau_low --range 0.05 0.35
    0.05
```

LISTING A.3: Individual policy runs and the ablation sweep.

A.4 Output Structure

Each run writes to its `--out` directory:

- `metrics.json` — aggregate metrics (VLM call rate, latency, throughput, peak memory, retention, SCE).
- `frame_log.csv` — per-frame drift components D_t , D_{embed} , D_{objects} , D_{track} , the chosen tier, and latency; the input consumed by the Semantic Observatory.
- `outputs/` — delivered VLM captions (cached or freshly generated) for qualitative inspection.

The aggregation script collates every run’s `metrics.json` into the comparison table. The Semantic Observatory (Section 4.5) is then run post hoc on any `frame_log.csv` to produce the semantic-evolution analytics, figures, and dashboard:

```
1 python scripts/run_observatory.py \
2     --frame_log experiments/proposed_threshold/frame_log.csv \
3     --output_dir results/observatory/proposed/ \
4     --experiment_name proposed_threshold
```

LISTING A.4: Post-hoc Semantic Observatory analysis.